
AssureX Claim Engine

Dual-Model AI Warranty Validation, Consensus Engine & Anti-Fraud Architecture

VERSION	1.0
TRACK	Aptech NextWave AI/ML
DATE	23 September 2026
AUTHOR / TEAM	AssureX Project Team

Table of Contents

1. Executive Summary & Problem Statement	4
Problem Statement	4
Proposed Solution	4
2. Consensus & Differencing Engine	5
3 Claim Classes	5
5 Model-Consistency Statuses	5
Confidence Difference Formula	5
Configurable Decision Matrix	5
3. System Architectural Diagrams & Flows	6
1. High-Level Component Architecture (Page 5 Flowchart)	6
2. DFD Level 0 (Context Diagram)	6
3. DFD Level 1 (Intake, OCR, Dual ML, Rule Engine, Reporting)	7
4. Use Case Diagram (Customer, Reviewer, Admin, Staff)	7
5. Activity Diagram (Intake -> OCR -> ML Consensus -> Rules -> Adjudication)	8
6. System Sequence Diagram	8
7. 8-Stage Claim Lifecycle State Machine	9
8. Master Decision Flowchart	9
9. Rule Engine & Policy Validation Flowchart	10
10. Dual-Model ML Consensus Pipeline	10
11. Python Tabular ML Pipeline	10
12. Google Teachable Machine Vision Architecture (600x400 Cards)	11
13. Complete Database ERD (All 13 Domain Entities)	11
14. System Deployment & Network Topology Diagram	11
4. Claim Lifecycle & Human Review Governance	12
5. Complete Data Dictionary	13
users	13
products	13
warranty_policies	14
product_warranties	14
claims	15
claim_documents	15
repair_histories	16
model_evaluations	16
rule_validation_logs	17
reviewer_actions	17
claim_status_histories	17
notifications	18
audit_logs	18
6. Performance & Benchmarking	19
Confusion Matrix - 225 Unseen Test Claims	19
Dual-Model Comparison	19

7. Security, Privacy & Compliance	20
Security Architecture Summary	20
8. Limitations & Future Enhancements	21
Future Enhancements	21
9. Requirements Traceability Snapshot	22
Appendix A. Project Metadata & Source Handling	23
Appendix B. Claim Classes, Statuses & Lifecycle Quick Reference	23

1. Executive Summary & Problem Statement

The AssureX Claim Engine is an enterprise-grade, dual-model artificial intelligence and rule-based warranty claim adjudication, validation, and anti-fraud platform. Engineered strictly in accordance with the official Aptech NextWave Software Requirements Specification (SRS Version 1.0) and the Page 5 architectural design, the system bridges the critical industry gap between manual, high-friction warranty processing and unexplainable, error-prone black-box automation. The platform provides end-to-end processing of commercial warranty claims through: Multi-Source Intake & Intelligent OCR: Ingests digital invoices, scanned receipts, and warranty cards across PDF, PNG, JPG, and JPEG formats, extracting entity metadata (Invoice number, serial number, purchase date, retailer, price) and calculating cryptographic SHA-256 document fingerprints. Dual-Model Consensus Architecture: Branch A (Tabular ML Classifier): An optimized Scikit-Learn Random Forest Classifier trained on preprocessed categorical and numerical claim vectors, evaluated across 5-fold cross-validation and achieving 100% accuracy on unseen test data across three target classes: Valid Claim, Invalid Claim, and Manual Review. Branch B (Vision ML Classifier): A standardized Google Teachable Machine (GTM) image-classification model evaluating dynamically generated 600 $\times$ 400 visual Claim Summary Cards, completely blinded to ground-truth labels and model outputs, achieving 100% test accuracy. Consensus & Confidence Differencing Engine: Computes absolute top-class confidence divergence ($\Delta_{\text{conf}} = |C_{\text{py}} - C_{\text{gtm}}|$) and maps dual predictions into one of five rigorous Model-Consistency Statuses: Strong Match, Acceptable Match, Weak Match, Model Disagreement, and Uncertain Result. Configurable Business Rule & Fraud Engine: Evaluates JSON-driven category policies (Consumer Electronics, Home Appliances, Industrial Tools), enforces coverage windows and grace periods, and triggers anomaly detection for chronological contradictions, serial number mismatches, unauthorized service facilities, and duplicate claim submissions. Human-in-the-Loop Workflow & Dynamic PDF Reporting: Manages an 8-stage claim lifecycle (Draft $\rightarrow$ Submitted $\rightarrow$ Under Evaluation $\rightarrow$ Additional Information Required $\rightarrow$ Manual Review $\rightarrow$ Approved $\rightarrow$ Rejected $\rightarrow$ Closed), providing dedicated Customer, Reviewer, and Admin portals, full decision override capability with audit trails, and automated downloadable PDF claim adjudication certificates. ---

Problem Statement

Traditional warranty claim evaluation requires personnel to inspect purchase details, product age, fault descriptions, repair history, warranty coverage, and supporting documents manually. This introduces processing delays, inconsistent decisions, missed exclusions, duplicate claims, and incomplete evidence handling. AssureX addresses these problems through document extraction, structured preprocessing, dual-model classification, configurable business rules, audit trails, and reviewer escalation.

Proposed Solution

Users submit claim information and supporting evidence through a web interface. The system validates and preprocesses the data, extracts information from documents, produces a standardized Claim Summary Card, runs the Python model on structured data, runs a separately trained Google Teachable Machine model on the visual card, compares their predictions and confidence, applies warranty policies, and produces a final Likely Valid, Likely Invalid, or Manual Review Required decision.

Primary architectural principle: the final claim decision is produced by the project's Python classification model, actual Google Teachable Machine model, warranty rule engine, and deterministic application logic rather than an external generative-AI API.

2. Consensus & Differencing Engine

AssureX uses exactly three claim classes and five model-consistency statuses. The two lists must remain conceptually separate throughout implementation, testing, reporting, and evaluation.

3 Claim Classes

Claim Class	Meaning
Valid Claim	Claim evidence and rules support warranty coverage.
Invalid Claim	Evidence or warranty rules indicate the claim is not covered.
Manual Review	Low confidence, disagreement, contradiction, or missing evidence requires a reviewer.

5 Model-Consistency Statuses

Status	Meaning / Routing
Strong Match	Classes match and confidence delta is at or below the strong-match threshold.
Acceptable Match	Classes match and confidence delta is within the acceptable-match band.
Weak Match	Classes match but confidence delta exceeds the acceptable-match threshold.
Model Disagreement	Python and GTM predict different claim classes.
Uncertain Result	Either model is below the configurable minimum confidence threshold.

Confidence Difference Formula

$\Delta_{conf} = |C_{py} - C_{gtm}|$
 where C_{py} is the Python model top-class confidence and C_{gtm} is the Google Teachable Machine top-class confidence.

Configurable Decision Matrix

Priority	Condition	Status
1	C_{py} or C_{gtm} below MIN_CONFIDENCE_THRESHOLD	Uncertain Result
2	Predicted classes differ	Model Disagreement
3	Classes match and $\Delta_{conf} \leq \text{STRONG_MATCH_DIFF}$	Strong Match
4	Classes match and $\text{STRONG_MATCH_DIFF} < \Delta_{conf} \leq \text{ACCEPTABLE_MATCH_DIFF}$	Acceptable Match
5	Classes match and $\Delta_{conf} > \text{ACCEPTABLE_MATCH_DIFF}$	Weak Match

Initial implementation thresholds are configurable and are implementation choices:
MIN_CONFIDENCE_THRESHOLD = 0.60, STRONG_MATCH_DIFF = 0.15, ACCEPTABLE_MATCH_DIFF = 0.30.

3. System Architectural Diagrams & Flows

The report represents all 14 mandatory diagrams as structured flow cards. These are intended to remain readable in print and to provide a direct bridge between the architecture specification and implementation evidence.

1. High-Level Component Architecture (Page 5 Flowchart)

01	Claim Details + Supporting Documents
	↓
02	Web-Based User Interface
	↓
03	Data Extraction and Validation
	↓
04	Python Data Pre-processing
	↓
05	Branch A: Structured Claim Data -> Python Classifier
	↓
06	Branch B: Claim Summary Card -> Google Teachable Machine
	↓
07	Prediction + Confidence Comparison
	↓
08	Warranty Rule Validation
	↓
09	Final Decision: Likely Valid / Likely Invalid / Manual Review Required

2. DFD Level 0 (Context Diagram)

01	Customer / Staff / Reviewer / Admin
	↓
02	-> AssureX Claim Engine
	↓
03	-> Authentication, Claim Intake, Evaluation, Review, Reporting
	↓
04	-> Database, Model Artifacts, Policy Files
	↓
05	-> Decisions, Alerts, Reports, Audit Records

3. DFD Level 1 (Intake, OCR, Dual ML, Rule Engine, Reporting)

01	Intake -> File Validation -> OCR / Text Extraction
	↓
02	OCR + User Data -> Verification -> Pre-processing
	↓
03	Pre-processing -> Python ML
	↓
04	Pre-processing -> Claim Summary Card -> GTM
	↓
05	Python + GTM -> Confidence Comparator
	↓
06	Comparator + Warranty Policies -> Rule Engine
	↓
07	Decision Engine -> Reviewer Queue / Final Decision / Reports

4. Use Case Diagram (Customer, Reviewer, Admin, Staff)

01	Customer: Register, Manage Profile, Register Product, Track Warranty, Submit Claim
	↓
02	Staff: Intake Claims, Upload Evidence, Correct Extracted Data, View Claim Status
	↓
03	Reviewer: Review Queue, Inspect Evidence, Override Decision, Request Information
	↓
04	Admin: Manage Policies, Monitor Anomalies, Analytics, Export, Audit

5. Activity Diagram (Intake -> OCR -> ML Consensus -> Rules -> Adjudication)

01	Start -> Submit Claim -> Validate Inputs -> Extract / Verify Documents
	↓
02	-> Pre-process -> Python Prediction + GTM Prediction
	↓
03	-> Compare Predictions / Confidence -> Apply Warranty Rules
	↓
04	-> Low Confidence or Conflict? -> Manual Review
	↓
05	-> Otherwise Final Decision -> Notify -> End

6. System Sequence Diagram

01	Browser -> Flask Controller: submit claim
	↓
02	Controller -> OCR Service: extract document fields
	↓
03	OCR Service -> Claim Service: verified claim payload
	↓
04	Claim Service -> Python Model: structured features
	↓
05	Claim Service -> GTM Model: Claim Summary Card
	↓
06	Models -> Comparator -> Rule Engine -> Decision Engine
	↓
07	Decision Engine -> Database -> Browser / Reviewer / Report Generator

7. 8-Stage Claim Lifecycle State Machine

01	Draft -> Submitted -> Under Evaluation
	↓
02	Under Evaluation -> Additional Information Required
	↓
03	Additional Information Required -> Under Evaluation
	↓
04	Under Evaluation -> Manual Review
	↓
05	Manual Review -> Approved
	↓
06	Manual Review -> Rejected
	↓
07	Approved -> Closed
	↓
08	Rejected -> Closed

8. Master Decision Flowchart

01	1. Evaluate Python and GTM predictions
	↓
02	2. Apply minimum confidence threshold
	↓
03	3. Determine class match / disagreement
	↓
04	4. Calculate $\Delta_{\text{conf}} = C_{\text{py}} - C_{\text{gtm}} $
	↓
05	5. Determine model-consistency status
	↓
06	6. Apply warranty policy, missing-document, duplicate and contradiction rules
	↓
07	7. Produce Likely Valid / Likely Invalid / Manual Review Required

9. Rule Engine & Policy Validation Flowchart

01	Load Category Policy -> Check Warranty Dates -> Check Fault Coverage
	↓
02	-> Proof of Purchase -> Serial Match -> Repair Conditions
	↓
03	-> Replacement Rules -> Mandatory Documents -> Duplicate / Contradiction Checks
	↓
04	-> HARD FAIL / WARNING / MANUAL REVIEW / PASS

10. Dual-Model ML Consensus Pipeline

01	Branch A: CSV Claim Record -> Preprocessor -> Python Classifier -> 3 probabilities
	↓
02	Branch B: Claim Record -> Summary Card -> Actual GTM Image Classifier -> 3 probabilities
	↓
03	Both -> Top Class + Top Confidence -> Delta + Status -> Decision Engine

11. Python Tabular ML Pipeline

01	Raw CSV -> Missing-Value Handling -> One-Hot Encoding
	↓
02	-> Scaling / Normalization -> Feature Engineering
	↓
03	-> Random Forest / HistGradientBoosting / MLP or Logistic Regression
	↓
04	-> Cross-Validation -> Metrics -> Best Model -> Joblib Artifact

12. Google Teachable Machine Vision Architecture (600x400 Cards)

01	Structured Claim Record
	↓
02	-> Standardized 600 x 400 Claim Summary Card
	↓
03	-> Two or more training variations per training claim
	↓
04	-> GTM image classes: Valid Claim / Invalid Claim / Manual Review
	↓
05	-> Exported model -> Backend inference -> 3-class confidence output

13. Complete Database ERD (All 13 Domain Entities)

01	User -> Products -> Product Warranties -> Claims
	↓
02	Claims -> Claim Documents / Repair Histories / Model Evaluations
	↓
03	Claims -> Rule Validation Logs / Reviewer Actions / Claim Status Histories
	↓
04	Users -> Notifications / Audit Logs
	↓
05	Warranty Policies -> Product Warranties

14. System Deployment & Network Topology Diagram

01	Client Browser (Desktop / Tablet / Mobile)
	↓
02	-> HTTPS Web Layer / Flask Application
	↓
03	-> Application Services -> SQLite Database / File Storage
	↓
04	-> Python ML Model Artifact + GTM Model Artifact + Policy Files
	↓
05	-> PDF/CSV Reporting + Audit Logs

4. Claim Lifecycle & Human Review Governance

The SRS defines an eight-stage claim lifecycle. Every transition is persisted through ClaimStatusHistory so the application can explain how a claim moved from draft intake through evaluation, manual review, approval or rejection, and closure.

Stage	Purpose
1. Draft	Claim is being prepared and may still be incomplete.
2. Submitted	Claim and evidence have been submitted for processing.
3. Under Evaluation	OCR, preprocessing, models, and rules are evaluating the claim.
4. Additional Information Required	Missing or conflicting evidence must be supplied.
5. Manual Review	Authorized reviewer evaluates uncertain or escalated cases.
6. Approved	Reviewer or deterministic workflow has approved the claim.
7. Rejected	Claim has been determined not to satisfy the applicable decision criteria.
8. Closed	Processing is complete and the claim is no longer active.

5. Complete Data Dictionary

The following data dictionary describes the 13 domain entities required by the current AssureX database model. Paragraph cells are explicitly width-constrained so long descriptions wrap safely without table overflow.

users

Table Name	Column Name	Data Type	Constraints	Description
users	id	INTEGER	PK, NOT NULL	Unique user identifier.
users	email	VARCHAR(255)	UNIQUE, NOT NULL	Login and notification email.
users	password_hash	VARCHAR(255)	NOT NULL	One-way password hash; plaintext is never stored.
users	role	VARCHAR(40)	NOT NULL	Customer, Staff, Reviewer, or Admin.
users	full_name	VARCHAR(160)	NOT NULL	Display name and contact identity.
users	phone	VARCHAR(40)	NULL	Optional contact number.
users	is_active	BOOLEAN	NOT NULL	Account availability flag.
users	created_at	DATETIME	NOT NULL	Account creation timestamp.
users	updated_at	DATETIME	NULL	Last profile update timestamp.

products

Table Name	Column Name	Data Type	Constraints	Description
products	id	INTEGER	PK, NOT NULL	Unique Product ID.
products	user_id	INTEGER	FK users.id, NOT NULL	Registered owner/customer.
products	name	VARCHAR(160)	NOT NULL	Product name.
products	category	VARCHAR(100)	NOT NULL	Warranty policy category.
products	brand	VARCHAR(100)	NOT NULL	Product manufacturer/brand.
products	model_number	VARCHAR(120)	NOT NULL	Product model identifier.
products	serial_number	VARCHAR(120)	UNIQUE, NOT NULL	Product serial number.
products	purchase_date	DATE	NOT NULL	Purchase date.
products	purchase_price	DECIMAL	NULL	Purchase amount.
products	retailer	VARCHAR(180)	NULL	Retailer or seller.
products	warranty_duration_months	INTEGER	NOT NULL	Nominal warranty duration.

warranty_policies

Table Name	Column Name	Data Type	Constraints	Description
warranty_policies	id	INTEGER	PK, NOT NULL	Unique policy identifier.
warranty_policies	category	VARCHAR(100)	UNIQUE, NOT NULL	Product category covered by the policy.
warranty_policies	coverage_duration_months	INTEGER	NOT NULL	Coverage duration.
warranty_policies	start_conditions	JSON/TEXT	NOT NULL	Conditions controlling warranty start.
warranty_policies	covered_faults	JSON/TEXT	NOT NULL	Fault categories covered.
warranty_policies	exclusions	JSON/TEXT	NOT NULL	Excluded damage/failure conditions.
warranty_policies	claim_reporting_period_days	INTEGER	NOT NULL	Maximum reporting period after fault.
warranty_policies	repair_conditions	JSON/TEXT	NOT NULL	Repair and service-center rules.
warranty_policies	mandatory_documents	JSON/TEXT	NOT NULL	Required claim documents.
warranty_policies	hard_fail_rules	JSON/TEXT	NOT NULL	Rules that force invalid/rejection treatment.
warranty_policies	warning_rules	JSON/TEXT	NOT NULL	Non-fatal warning conditions.
warranty_policies	manual_review_rules	JSON/TEXT	NOT NULL	Escalation rules.

product_warranties

Table Name	Column Name	Data Type	Constraints	Description
product_warranties	id	INTEGER	PK, NOT NULL	Warranty record identifier.
product_warranties	product_id	INTEGER	FK products.id, NOT NULL	Linked product.
product_warranties	policy_id	INTEGER	FK warranty_policies.id, NOT NULL	Applied policy.
product_warranties	provider	VARCHAR(180)	NOT NULL	Warranty provider.
product_warranties	start_date	DATE	NOT NULL	Warranty start date.
product_warranties	expiry_date	DATE	NOT NULL	Warranty expiry date.
product_warranties	coverage_conditions	TEXT	NOT NULL	Coverage notes/conditions.
product_warranties	extended	BOOLEAN	NOT NULL	Extended-warranty indicator.
product_warranties	status	VARCHAR(40)	NOT NULL	Active, Expired, Approaching Expiry, etc.

claims

Table Name	Column Name	Data Type	Constraints	Description
claims	id	INTEGER	PK, NOT NULL	Internal claim record ID.
claims	claim_id	VARCHAR(64)	UNIQUE, NOT NULL	Public Claim ID.
claims	user_id	INTEGER	FK users.id, NOT NULL	Claimant.
claims	product_id	INTEGER	FK products.id, NOT NULL	Claimed product.
claims	warranty_id	INTEGER	FK product_warranties.id, NOT NULL	Applicable warranty.
claims	status	VARCHAR(50)	NOT NULL	One of the 8 SRS lifecycle stages.
claims	fault_date	DATE	NOT NULL	Date fault occurred.
claims	fault_description	TEXT	NOT NULL	Reported fault description.
claims	damage_type	VARCHAR(100)	NULL	Damage classification.
claims	submission_date	DATE	NULL	Claim submission date.
claims	final_decision	VARCHAR(60)	NULL	Likely Valid, Likely Invalid, or Manual Review Required.
claims	decision_explanation	TEXT	NULL	Human-readable decision rationale.
claims	created_at	DATETIME	NOT NULL	Claim creation timestamp.
claims	updated_at	DATETIME	NULL	Last claim update timestamp.

claim_documents

Table Name	Column Name	Data Type	Constraints	Description
claim_documents	id	INTEGER	PK, NOT NULL	Document record identifier.
claim_documents	claim_id	INTEGER	FK claims.id, NOT NULL	Owning claim.
claim_documents	document_type	VARCHAR(80)	NOT NULL	Receipt, warranty card, image, repair report, etc.
claim_documents	original_filename	VARCHAR(255)	NOT NULL	Uploaded filename.
claim_documents	storage_path	VARCHAR(500)	NOT NULL	Server-side storage location.
claim_documents	mime_type	VARCHAR(120)	NOT NULL	Validated file MIME type.
claim_documents	sha256	CHAR(64)	INDEX, NOT NULL	Cryptographic duplicate fingerprint.
claim_documents	ocr_text	TEXT	NULL	Extracted document text.
claim_documents	extracted_data	JSON/TEXT	NULL	Parsed fields such as invoice date and serial.
claim_documents	verified	BOOLEAN	NOT NULL	Whether extracted data was reviewed.
claim_documents	uploaded_at	DATETIME	NOT NULL	Upload timestamp.

repair_histories

Table Name	Column Name	Data Type	Constraints	Description
repair_histories	id	INTEGER	PK, NOT NULL	Repair history identifier.
repair_histories	product_id	INTEGER	FK products.id, NOT NULL	Product repaired.
repair_histories	claim_id	INTEGER	FK claims.id, NULL	Related claim if applicable.
repair_histories	repair_date	DATE	NOT NULL	Repair date.
repair_histories	service_center	VARCHAR(180)	NOT NULL	Repair center.
repair_histories	parts_replaced	TEXT	NULL	Replaced parts.
repair_histories	outcome	TEXT	NULL	Repair outcome.
repair_histories	repair_cost	DECIMAL	NULL	Repair cost.
repair_histories	is_authorized_center	BOOLEAN	NOT NULL	Authorized/unauthorized service indicator.

model_evaluations

Table Name	Column Name	Data Type	Constraints	Description
model_evaluations	id	INTEGER	PK, NOT NULL	Evaluation identifier.
model_evaluations	claim_id	INTEGER	FK claims.id, NOT NULL	Evaluated claim.
model_evaluations	python_predicted_class	VARCHAR(60)	NOT NULL	Python top class.
model_evaluations	python_conf_valid	FLOAT	NOT NULL	Python probability for Valid Claim.
model_evaluations	python_conf_invalid	FLOAT	NOT NULL	Python probability for Invalid Claim.
model_evaluations	python_conf_manual_review	FLOAT	NOT NULL	Python probability for Manual Review.
model_evaluations	gtm_predicted_classes	VARCHAR(60)	NULL	GTM top class.
model_evaluations	gtm_conf_valid	FLOAT	NULL	GTM probability for Valid Claim.
model_evaluations	gtm_conf_invalid	FLOAT	NULL	GTM probability for Invalid Claim.
model_evaluations	gtm_conf_manual_review	FLOAT	NULL	GTM probability for Manual Review.
model_evaluations	confidence_difference	FLOAT	NULL	Absolute top-class confidence delta.
model_evaluations	consistency_status	VARCHAR(40)	NULL	One of the 5 model-consistency statuses.
model_evaluations	python_model_version	VARCHAR(80)	NULL	Python artifact/model version.
model_evaluations	gtm_model_version	VARCHAR(80)	NULL	GTM artifact/model version.
model_evaluations	evaluated_at	DATETIME	NOT NULL	Evaluation timestamp.

rule_validation_logs

Table Name	Column Name	Data Type	Constraints	Description
rule_validation_logs	id	INTEGER	PK, NOT NULL	Rule evaluation log identifier.
rule_validation_logs	claim_id	INTEGER	FK claims.id, NOT NULL	Related claim.
rule_validation_logs	policy_category	VARCHAR(100)	NOT NULL	Policy category used.
rule_validation_logs	rule_code	VARCHAR(100)	NOT NULL	Stable rule identifier.
rule_validation_logs	result	VARCHAR(20)	NOT NULL	PASS, WARN, or FAIL.
rule_validation_logs	message	TEXT	NOT NULL	Rule explanation.
rule_validation_logs	evidence	JSON/TEXT	NULL	Evidence used by the rule.
rule_validation_logs	created_at	DATETIME	NOT NULL	Rule evaluation timestamp.

reviewer_actions

Table Name	Column Name	Data Type	Constraints	Description
reviewer_actions	id	INTEGER	PK, NOT NULL	Reviewer action identifier.
reviewer_actions	claim_id	INTEGER	FK claims.id, NOT NULL	Reviewed claim.
reviewer_actions	reviewer_id	INTEGER	FK users.id, NOT NULL	Reviewer performing action.
reviewer_actions	action	VARCHAR(60)	NOT NULL	Approve, Reject, Request Information, Override, etc.
reviewer_actions	previous_decision	VARCHAR(60)	NULL	Automated or prior decision.
reviewer_actions	new_decision	VARCHAR(60)	NULL	Reviewer decision.
reviewer_actions	comments	TEXT	NULL	Reviewer explanation.
reviewer_actions	created_at	DATETIME	NOT NULL	Action timestamp.

claim_status_histories

Table Name	Column Name	Data Type	Constraints	Description
claim_status_histories	id	INTEGER	PK, NOT NULL	Status history row identifier.
claim_status_histories	claim_id	INTEGER	FK claims.id, NOT NULL	Related claim.
claim_status_histories	old_status	VARCHAR(50)	NULL	Previous lifecycle status.
claim_status_histories	new_status	VARCHAR(50)	NOT NULL	New lifecycle status.
claim_status_histories	changed_by	INTEGER	FK users.id, NULL	User/system actor.
claim_status_histories	reason	TEXT	NULL	Reason for transition.
claim_status_histories	created_at	DATETIME	NOT NULL	Transition timestamp.

notifications

Table Name	Column Name	Data Type	Constraints	Description
notifications	id	INTEGER	PK, NOT NULL	Notification identifier.
notifications	user_id	INTEGER	FK users.id, NOT NULL	Notification recipient.
notifications	claim_id	INTEGER	FK claims.id, NULL	Related claim when applicable.
notifications	product_id	INTEGER	FK products.id, NULL	Related product when applicable.
notifications	notification_type	VARCHAR(80)	NOT NULL	Warranty expiry, status change, action request, etc.
notifications	message	TEXT	NOT NULL	User-facing notification text.
notifications	is_read	BOOLEAN	NOT NULL	Read/unread state.
notifications	created_at	DATETIME	NOT NULL	Creation timestamp.

audit_logs

Table Name	Column Name	Data Type	Constraints	Description
audit_logs	id	INTEGER	PK, NOT NULL	Audit event identifier.
audit_logs	user_id	INTEGER	FK users.id, NULL	Actor, if user initiated.
audit_logs	action	VARCHAR(120)	NOT NULL	Audited action name.
audit_logs	entity_type	VARCHAR(80)	NULL	Affected entity type.
audit_logs	entity_id	VARCHAR(80)	NULL	Affected entity ID.
audit_logs	details	JSON/TEXT	NULL	Structured event details.
audit_logs	ip_address	VARCHAR(64)	NULL	Originating IP when available.
audit_logs	created_at	DATETIME	NOT NULL	Audit timestamp.

6. Performance & Benchmarking

The benchmark table is populated from the project report / model-evidence sources when explicit values are available. The known reported Random Forest result is 100.0%. Values not present in the source are intentionally labeled as not reported rather than fabricated.

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Model' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=7.6, greek=0, italic=0, link=[], rise=0, text='Model', textColor=Color(.058824,.090196,.164706,1), us_lines=[])] 'style': <ParagraphStyle 'TableHeader'> 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Evaluation' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=7.6, greek=0, italic=0, link=[], rise=0, text='Evaluation', textColor=Color(.058824,.090196,.164706,1), us_lines=[])] 'style': <ParagraphStyle 'TableHeader'> 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Status' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=7.6, greek=0, italic=0, link=[], rise=0, text='Status', textColor=Color(.058824,.090196,.164706,1), us_lines=[])] 'style': <ParagraphStyle 'TableHeader'> 'bulletText': None 'debug': 0) #Paragraph</pre>
Random Forest	100%	Selected baseline / best-known result
HistGradientBoosting	99.1%	Compared model
MLP / Logistic Regression	94.6%	Compared baseline

Confusion Matrix - 225 Unseen Test Claims

Actual \ Predicted	Valid	Invalid	Manual Review	Total
Valid	75	0	0	75
Invalid	0	75	0	75
Manual Review	0	0	75	75
Total	75	75	75	225

Dual-Model Comparison

Metric	Reported Value
Unseen claims evaluated	36
Python/GTM agreement	100%
Mean top-class confidence delta	0.0088

Interpretation: the report should use only measured benchmark values that are actually present in the project's evidence artifacts. A high benchmark result does not replace the need for held-out testing, class-wise metrics, model comparison, and manual-review routing.

7. Security, Privacy & Compliance

Control	Implementation / Purpose
RBAC	Role-based access for Customer, Staff, Reviewer, and Administrator.
Authentication	Secure password hashing, session control, authentication and authorization checks.
File Validation	Validate extension, MIME type, file size, and required document categories before storage.
SHA-256 Document Hashing	Fingerprint uploaded documents to identify duplicate evidence across claims.
Audit Logging	Append-oriented audit events for account actions, uploads, predictions, status changes, reviews, and decisions.
PII Minimization	Store only the customer/document fields necessary for processing and evaluation.
Privacy Protection	Treat receipts, warranty records, serial numbers and contact data as protected personal/business information.
GDPR-aligned Principles	Use purpose limitation, data minimization, access control, retention awareness, and traceability. This is not a legal certification.
Injection/Abuse Controls	Validate inputs, protect routes, avoid unsafe dynamic queries, and test unauthorized API access.

Security Architecture Summary

01	Authenticated user / evaluator	
		↓
02	Role-based authorization	
		↓
03	Validated claim/document intake	
		↓
04	Protected application services and database	
		↓
05	Model + rule evaluation with audit trail	
		↓
06	Reviewer override with retained original decision evidence	
		↓
07	Controlled exports and reports	

8. Limitations & Future Enhancements

Limitation	Impact
OCR resolution caveats	Image quality, skew, blur, lighting, handwriting, and unusual receipt layouts can reduce OCR extraction quality. Human verification remains important.
Cloud storage scaling	SQLite and local file storage are portable for a competition demonstration but should be replaced by managed database/object storage for larger multi-tenant production workloads.
Synthetic data realism	A synthetic dataset is useful for controlled evaluation but cannot represent every real-world warranty pattern or document variation.
Model drift	Real claim distributions may change; periodic retraining and validation would be required in a production deployment.
Policy evolution	Warranty policies change by product, region, contract version, and business partner; policy files must remain configurable.

Future Enhancements

- Cloud object storage and managed PostgreSQL for production-scale deployment.
- Asynchronous document processing and queue-based OCR for large claim volumes.
- Advanced document parsing and OCR confidence scoring.
- Versioned policy authoring and approval workflows.
- Optional LLM-assisted warranty text interpretation with strict separation from final claim decision logic.
- Model monitoring, drift detection, recalibration, and automated retraining pipelines.
- Additional analytics for fraud patterns, repair networks, and warranty leakage.

Important architecture constraint: future generative-AI capabilities must not replace the deterministic final claim decision path required by the project specification.

9. Requirements Traceability Snapshot

Requirement Area	Mapped Capability
Authentication & RBAC	Users, roles, protected routes and user profiles
Product & Warranty Management	Products, warranties, configurable policies and expiry alerts
Document Operations	PDF/image upload, OCR, extraction, verification, secure hashing
Claim Intake	Claim registration, evidence upload, repair history and validation
Python ML	Preprocessing, feature engineering, model training, confidence scores
Google Teachable Machine	Claim Summary Cards, GTM training, confidence scores and independent evaluation
Model Comparison	Class match, Delta_conf, five consistency statuses
Rule Engine	Warranty rules, exclusions, mandatory documents and review rules
Fraud / Consistency	Duplicate claims, duplicate documents, contradictions and serial checks
Human Review	Queue, comments, overrides and audit history
Reporting	PDF reports, CSV/Excel export, dashboards and analytics
Governance	Audit logs, model versions, AI_USAGE.md and genuine Git history

Appendix A. Project Metadata & Source Handling

This PDF generator is designed to run from the AssureX project repository. When documentation/PROJECT_REPORT.md exists, the script reads its Executive Summary, Problem Statement, Proposed Solution, and related headings where available. The report also contains explicit built-in architectural tables for the mandatory system artifacts so the PDF remains deterministic and complete.

Item	Value
Project	AssureX Claim Engine
Primary source	documentation/PROJECT_REPORT.md
Supplementary model evidence	documentation/PYTHON_MODEL_EVIDENCE.md when present
Output	reports/AssureX_Project_Report.pdf
Page size	A4 portrait
Primary renderer	ReportLab

Appendix B. Claim Classes, Statuses & Lifecycle Quick Reference

Dimension	Values
Claim Classes	Valid Claim Invalid Claim Manual Review
Model-Consistency Statuses	Strong Match Acceptable Match Weak Match Model Disagreement Uncertain Result
Claim Lifecycle	Draft -> Submitted -> Under Evaluation -> Additional Information Required -> Manual Review -> Approved / Rejected -> Closed